

Technologies Utilisees

Djezzy Supervision Reseau

Systeme NOC de gestion des reclamations
et supervision des sites BTS

PFE 2025-2026

Equipe: Belakebi Amine & Bouachach Amel

SOMMAIRE

1	Langages de programmation JavaScript, Python, CSS3, HTML5, SQL
2	Frameworks Frontend React, React Router, Recharts, Leaflet, Axios...
3	Frameworks Backend Django, DRF, SimpleJWT, OpenAI SDK...
4	Base de donnees PostgreSQL, Django ORM, psycopg2, Pillow
5	Navigateur web Mozilla Firefox - outils de debug, compatibilite
6	Outils de developpement VS Code, Git/GitHub, npm, pip, CRA, draw.io
7	Technologies web & mapping PWA, OpenStreetMap, Leaflet, HTML5
8	Intelligence artificielle Mistral AI, algorithme Jaccard, fallback local
9	Securite JWT, HSTS, CORS, Rate Limiting, DOMPurify...

LANGAGES DE PROGRAMMATION

Les langages utilises pour developper le frontend, le backend et la base de donnees

JavaScript (ES6+) 2024+

JavaScript est le langage principal du frontend. On l'a choisi car c'est le seul langage nativement supporte par tous les navigateurs web modernes. Les features ES6+ (arrow functions, async/await, destructuring, template literals, optional chaining) rendent le code plus lisible et maintenable. C'est aussi le langage le plus utilise au monde pour le developpement web, ce qui facilite la reutilisation de code et l'accès à une enorme communauté.

Python 3.10+

Python est le langage du backend, utilise avec Django. On l'a choisi pour sa simplicité de syntaxe, sa grande bibliothèque de modules (pillow, psycopg2, openai), et son excellente integration avec Django. Python est le langage standard pour les projets web avec Django/DRF, et il offre également de très bonnes capacités pour l'intelligence artificielle (Mistral AI SDK). Sa courbe d'apprentissage douce facilite aussi la maintenance du code.

CSS3

CSS3 est utilise pour le styling du frontend React. On a fait le choix d'utiliser du CSS natif (sans frameworks comme Tailwind ou SASS) avec des CSS Custom Properties (variables) pour créer un système de thèmes centralisé. Les animations @keyframes et les transitions permettent d'ajouter du dynamisme à l'interface. C'est le choix le plus simple et le plus direct pour styler des composants React, sans dépendance supplémentaire.

HTML5

HTML5 est le langage de structure utilise dans le fichier index.html d'entrée React et également dans la generation de rapports IA (templates HTML dynamiques). Les elements sémantiques HTML5 (section, header, nav) et les APIs modernes (Geolocation pour la carte, localStorage pour les tokens JWT) sont essentiels au bon fonctionnement de l'application. C'est le fondement de tout projet web moderne.

SQL

SQL est utilise indirectement via les migrations Django ORM et les requetes PostgreSQL. Django genere automatiquement les requetes SQL a partir des modeles Python, mais la comprehension de SQL reste necessaire pour optimiser les performances, creer des index, et diagnostiquer les problemes de requetes. Les 25 fichiers de migrations du projet reflektent le schema SQL genere pour les 6 tables de la base de donnees.

FRAMEWORKS FRONTEND

Les bibliothèques et frameworks utilisés pour construire l'interface utilisateur

React 19.2.4

React est le framework UI principal du frontend. On l'a choisi pour sa popularité énorme, son écosystème riche, et son architecture en composants qui facilite la réutilisation du code. React 19 offre des performances optimisées et des hooks modernes (useState, useEffect, useContext). Le virtual DOM de React permet des mises à jour efficaces sans rechargement de page. C'est le framework le plus populaire au monde avec la plus grande communauté.

React Router DOM 6.14.0

React Router gère la navigation côté client (single page application). On l'a utilisé pour créer des routes séparées pour chaque page (Login, AdminDashboard, EngineerDashboard, SupervisorDashboard, CallCenter, Profile) avec protection par rôle via PrivateRoute. C'est le routing standard pour React, simple à configurer et bien documenté.

Recharts 3.8.1

Recharts est une bibliothèque de graphiques React pour la visualisation de données. On l'a choisie pour les KPIs et tableaux de bord car elle est composée de composants React natifs (facile à intégrer), supporte les graphiques en aires, barres, camemberts, et lignes, et offre un thème personnalisable. C'est la solution la plus simple pour ajouter des graphiques à un dashboard React.

Leaflet / React Leaflet 9.4 / 5.0.0

Leaflet est une bibliothèque JavaScript pour cartes interactives, utilisée ici pour afficher la position des sites BTS sur une carte OpenStreetMap. React Leaflet fournit les composants React (MapContainer, TileLayer, Marker, Popup, Circle) pour intégrer Leaflet proprement. On l'a choisie car c'est open source, léger (< 42KB), et offre toutes les fonctionnalités nécessaires (markers, popups, cercles, zoom). Leaflet est la lib de cartes web la plus populaire au monde.

Axios 1.14.0

Axios est un client HTTP pour JavaScript. On l'a utilisé dans la page Login pour les requêtes d'authentification JWT, car il offre une gestion automatique des headers, le support des intercepteurs (utile pour le refresh automatique des tokens), et une API plus propre que le fetch natif. Les autres pages utilisent le fetch API natif pour rester léger, mais Axios est présent pour les cas complexes.

DOMPurify 3.4.11

DOMPurify est un sanitiseur HTML/JS. On l'a intégré pour sécuriser l'affichage de contenu HTML généré par Mistral AI (rapports IA). Sans DOMPurify, un contenu malveillant injecté dans les rapports pourrait exécuter du JavaScript (XSS). C'est la bibliothèque de référence pour la sanitisation HTML, utilisée par Mozilla, Google et GitHub.

html2pdf.js 0.14.0

html2pdf.js convertit du HTML en PDF directement côté client. On l'a utilisé pour permettre aux utilisateurs de télécharger les rapports IA et les données en PDF depuis le navigateur. C'est une solution simple qui évite de passer par le backend pour la génération de PDF côté client.

FRAMEWORKS BACKEND

Les frameworks et bibliotheques Python utilises pour l'API REST et la logique metier

Django 6.0.3

Django est le framework web Python principal du backend. On l'a choisi pour son architecture MVT (Model-View-Template) robuste, son ORM puissant, son admin automatique, et sa securite integree (CSRF, XSS, SQL injection). Django est le framework Python le plus mature et le plus utilise pour les projets web professionnels. Sa convention 'batteries included' fournit tout ce dont on a besoin : authentication, migrations, formulaires, et routage.

Django REST Framework (DRF)

DRF est le framework standard pour creer des APIs REST avec Django. On l'a utilise pour construire les endpoints JSON (/api/accounts/, /api/sites/, /api/reclamations/, etc.). DRF fournit les serialiseurs pour convertir les modeles Django en JSON, les vues generiques (ListCreateAPIView, RetrieveUpdateDestroyAPIView), le routage automatique, et le systeme de permissions et throttling. C'est l'outil de reference pour les APIs REST en Python/Django.

django-rest-framework-simplejwt

SimpleJWT implemente l'authentification JWT (JSON Web Tokens) pour DRF. On l'a choisi car c'est la solution la plus complete pour JWT dans Django : tokens d'accès (1h) + refresh (7j), rotation des refresh tokens, blacklist pour la deconnexion securisee, et support des claims custom (on injecte le role et le code_user dans le token). JWT est standard pour les APIs REST modernes et separe bien le frontend du backend.

django-cors-headers 4.3.0

django-cors-headers gere les en-tetes CORS (Cross-Origin Resource Sharing) entre le frontend React (localhost:3000) et le backend Django (localhost:8000). Sans ce module, le navigateur bloquerait les requetes API du frontend a cause de la politique Same-Origin. On l'a configure avec une liste blanche restrictive (4 origines localhost) plutot que CORS_ALLOW_ALL_ORIGINS=True pour maintenir la securite.

OpenAI SDK (compatible Mistral AI)

Le SDK OpenAI est utilise pour integrer Mistral AI (mistral-small-latest) via l'API compatible OpenAI d Mistral (api.mistral.ai/v1). On a choisi Mistral car c'est un LLM europeen performant, avec un excellent rapport qualite/prix. Il genere les titres et descriptions des groupes de tickets, et produit les rapports IA HTML. Le SDK OpenAI compatible evite d'ecrire du code d'appel API specifique a Mistral.

python-decouple

python-decouple gere les variables d'environnement depuis le fichier .env. On l'a utilise pour separer la configuration sensible (SECRET_KEY, DB_PASSWORD, API keys) du code source. C'est un standard de securite : ne jamais committer les secrets dans Git. Simples et leger, il fonctionne avec un fichier .env et des variables d'environnement systeme.

Pillow

12.1.1

Pillow est la bibliotheque Python de reference pour le traitement d'images. On l'a utilise pour gerer les fichiers media (avatars, images de sites) dans Django. C'est un prerequis de Django pour les ImageField et FileField, et la seule bibliotheque d'image mature pour Python.

BASE DE DONNEES

Le système de gestion de base de données et les outils associés

PostgreSQL 15

PostgreSQL est le SGBD (Système de Gestion de Base de Données) relationnel utilisé. On l'a choisi pour sa fiabilité, ses performances avec de gros volumes de données, son support ACID complet, et sa conformité aux standards SQL. C'est la base de données open source la plus avancée au monde, recommandée par Django pour les projets professionnels. Elle gère les 6 tables du projet avec 25 migrations.

Django ORM

Django ORM est l'outil de mappage objet-relationnel (ORM) intégré à Django. Il permet d'écrire des requêtes en Python au lieu de SQL brut. On l'a utilisé pour définir les 6 modèles (CustomUser, Reclamation, GroupeTicket, SiteReseau, CommentaireTicket, RapportIA) avec leurs relations FK, valider les données, et générer automatiquement les migrations SQL. C'est l'ORM le plus simple et le plus productif pour Django.

psycopg2 2.9.11

psycopg2 est le connecteur PostgreSQL pour Python. C'est le driver recommandé par Django pour se connecter à PostgreSQL. C'est un module C rapide et fiable qui gère toutes les opérations SQL. Il est utilisé en arrière-plan par Django ORM chaque fois qu'une requête est envoyée à la base de données.

NAVIGATEUR WEB

Le navigateur utilise pour le developpement et les tests

Mozilla Firefox 128+

Firefox est le navigateur principal utilise pendant le developpement et les tests de l'application. On l'a choisi pour plusieurs raisons :

1. Outils de developpement integres : Les Firefox DevTools offrent un inspecteur DOM, un debogueur JavaScript, un onglet Reseau (Network), et un onglet Stockage (localStorage, cookies) tres performants pour debugger les appels API et le stockage JWT.
2. Compatibilite W3C : Firefox est connu pour etre le navigateur le plus conforme aux standards W3C. Tester sur Firefox garantit que l'application respecte les normes web.
3. Rendu CSS different : Firefox utilise son propre moteur de rendu (Gecko), different de Chromium (Blink). Tester sur Firefox permet de detecter les incompatibilites CSS qui passeraient inaperues sur Chrome.
4. Respect de la vie privree : Firefox est developpe par Mozilla, une organisation sans but lucratif dediee a l'ouverture du web. C'est le navigateur le plus respectueux de la vie privree, sans telemetrie par default.
5. Open source : Firefox est 100% open source, ce qui est coherent avec l'utilisation d'autres outils open source du projet (Django, PostgreSQL, Leaflet).

OUTILS DE DEVELOPPEMENT

Les éditeurs, systèmes de gestion de version et outils de build utilisés

Visual Studio Code

VS Code est l'IDE (éditeur de code) principal utilisé pour développer le projet. On l'a choisi pour sa légèreté, son support multi-langages (Python, JavaScript, CSS, HTML), ses extensions puissantes (Python, ESLint, Prettier, GitLens), et son terminal intégré. Le fichier `.vscode/settings.json` configure l'interpréteur Python et les chemins d'analyse. C'est l'IDE le plus populaire au monde selon les enquêtes Stack Overflow.

Git / GitHub

Git est le système de gestion de version utilisé, avec GitHub comme dépôt distant (github.com/Belakebi-Amine/djezzy_supervision). On a choisi Git car c'est le standard absolu du développement logiciel moderne, et GitHub offre l'hébergement, les pull requests, et la collaboration. Les branches permettent de travailler séparément sur le frontend et le backend, et le `.gitignore` protège les fichiers sensibles (`.env`, `venv/`, `node_modules/`).

npm

npm (Node Package Manager) gère les dépendances JavaScript du frontend React. C'est le gestionnaire de paquets par défaut de Node.js, utilisé pour installer et mettre à jour les 12 bibliothèques du frontend (react, recharts, leaflet, axios...). Le fichier `package-lock.json` garantit des installations reproductibles.

pip / Python venv

pip est le gestionnaire de paquets Python pour installer les dépendances backend (django, django-rest-framework, psycopg2...). Python venv crée un environnement virtuel isolé (dossier `venv/`) pour éviter les conflits de versions entre projets. C'est la pratique standard en développement Python.

Create React App (CRA)^{react-scripts 5.0.1}

CRA est l'outil de build du frontend React. Il gère automatiquement Webpack, Babel, ESLint, et le hot-reloading pendant le développement. On l'a utilisé pour initialiser le projet React et bénéficier d'une configuration optimisée sans avoir à configurer Webpack manuellement. C'est l'outil recommandé par React pour démarrer un projet.

draw.io (diagrams.net)

draw.io est l'outil de diagrammation UML utilise pour la conception du projet. On l'a utilise pour creer 3 diagrammes dans le dossier Conception/ : diagramme de cas d'utilisation (USE_CASE.drawio), diagramme de sequence (diag_Seq.drawio), et diagramme de classes (diag_Classe.drawio). C'est un outil gratuit, open source, qui fonctionne dans le navigateur sans installation, et exporte en PNG/PDF.

ESLint

ESLint est l'outil de linting JavaScript integre dans CRA. Il detecte automatiquement les erreurs de syntaxe, le code mort, et les problemes de style dans le code JavaScript. La configuration dans package.json utilise les presets react-app et react-app/jest. C'est l'outil standard pour maintenir la qualite du code JavaScript/React.

TECHNOLOGIES WEB ET MAPPING

Les technologies specifiques au web et a la cartographie interactive

Progressive Web App (PWA)

L'application est configuree comme une PWA via le fichier manifest.json. Cela permet de l'installer comme une application native sur les appareils, de fonctionner en mode hors-ligne (avec cache), et d'offrir une experience utilisateur proche d'une app mobile. C'est un standard web moderne qui evite de developper une app mobile separee.

OpenStreetMap

OpenStreetMap (OSM) est le fournisseur de tuiles cartographiques utilise avec Leaflet. On l'a choisi car c'est gratuit, open source, et offre une couverture mondiale. Contrairement a Google Maps, OSM ne necessite pas de cle API ni de restrictions de quota. Les donnees sont community-driven et mises a jour regulierement. C'est le standard pour les projets open source utilisant des cartes.

localStorage (Web Storage API)

localStorage est utilise côté client pour stocker les tokens JWT (access + refresh) de maniere persistante. Contrairement aux cookies, localStorage n'est pas envoye automatiquement avec chaque requete HTTP, ce qui donne plus de controle sur l'envoi des tokens (via les headers Authorization). C'est le stockage standard pour les apps JWT modernes.

Fetch API / XHR

Fetch API est l'API native des navigateurs pour effectuer des requetes HTTP. On l'a utilisee pour la plupart des appels API du frontend (tickets, dashboard, sites). Elle est plus moderne et legere que XMLHttpRequest (XHR). Axios est aussi present pour les cas complexes (intercepteurs JWT). Les deux coexistent dans le projet.

INTELLIGENCE ARTIFICIELLE

Les technologies IA integrees dans l'application

Mistral AI (mistral-small-latest)

Mistral AI est le modele de langage (LLM) utilise pour l'intelligence artificielle du projet. On l'a choisi car :

1. Performance : Mistral-small est un modele performant avec un excellent rapport qualite/prix par rapport a GPT-4.
2. API compatible OpenAI : L'API de Mistral est compatible avec le SDK OpenAI, ce qui simplifie l'integration (meme syntaxe, meme structure de reponse).
3. Souverainete europeenne : Mistral est une entreprise francaise, ce qui est important pour la conformite RGPD et la souverainete des donnees.
4. Utilisations dans le projet :
 - Generation automatique des titres et descriptions pour les groupes de tickets
 - Generation de rapports HTML d'analyse reseau avec la section 'commentaire IA'
 - Analyse des tendances et patterns dans les reclamations

Algorithme Jaccard (fallback local)

L'algorithme de similarite de Jaccard est utilise comme fallback quand Mistral AI est indisponible. Il compare les mots-cles des tickets et calcule un score de similarite ($\geq 80\%$ de similarite = meme groupe). Les tickets du meme site et de la meme fenetre de 7 jours sont regroupees. C'est une solution de secours simple mais efficace qui assure la continuite du service meme sans connexion a l'API IA.

SECURITE

Les mecanismes de securite implements dans l'application

JWT (JSON Web Tokens) - SimpleJWT

L'authentification JWT est implementee via SimpleJWT avec un systeme de tokens d'accès (1h) et refresh (7j). Les refresh tokens sont rotates a chaque utilisation et blacklistes apres rotation ou deconnexion. Des claims custom (role, code_user) sont injectes dans le token pour la gestion des permissions côté client. Le logout blackliste le refresh token pour empêcher toute reutilisation. C'est le standard d'authentification pour les APIs REST modernes.

HSTS (HTTP Strict Transport Security)

HSTS est un mecanisme de securite qui force le navigateur a communiquer uniquement en HTTPS. Configuré en production avec :

- SECURE_HSTS_SECONDS = 31536000 (1 an)
- SECURE_HSTS_INCLUDE_SUBDOMAINS = True
- SECURE_HSTS_PRELOAD = True

Pourquoi HSTS est essentiel :

1. Protection SSL stripping : Empêche les attaques où un attaquant force la connexion HTTP au lieu de HTTPS pour intercepter les données.
2. Protection contre les cookies non securises : Avec SESSION_COOKIE_SECURE=True et CSRF_COOKIE_SECURE=True, les cookies ne sont envoyes qu'en HTTPS.
3. Preload : Le site peut être ajoute a la HSTS preload list du navigateur, ce qui force HTTPS dès la première visite.

En developpement (DEBUG=True), HSTS est desactive (SECURE_HSTS_SECONDS=0) pour permettre le developpement local sans certificat SSL.

CORS (Cross-Origin Resource Sharing)

CORS est configure via django-cors-headers avec une liste blanche restrictive : localhost:3000, 127.0.0.1:3000, localhost:5173, 127.0.0.1:5173. CORS_ALLOW_ALL_ORIGINS=False empêche toute origine non listee d'accéder a l'API. C'est un mecanisme de securite essentiel car le frontend et le backend sont sur des ports differents (3000 vs 8000), ce qui constitue une cross-origin request.

Rate Limiting (Throttling)

Le rate limiting est implemente via DRF throttling : 30 requetes/minute pour les utilisateurs anonymes et 100 requetes/minute pour les utilisateurs connectes. Cela protege l'API contre les attaques par deni de service (DDoS) et le scraping. C'est une securite de base indispensable pour toute API publique.

DOMPurify - Protection XSS

DOMPurify sanitise le HTML genere par Mistral AI avant de l'afficher dans le navigateur. Sans cette protection, un prompt injection pourrait injecter du JavaScript malveillant dans les rapports IA (attaques XSS - Cross-Site Scripting). DOMPurify supprime tous les scripts, events handlers, et elements dangereux du HTML. C'est la bibliotheque de reference utilisee par Mozilla, Google et GitHub.

Restriction IP Admin (Middleware)

Un middleware personnalise (AdminIPRestrictionMiddleware) bloque l'accès a /admin/ sauf depuis les IP autorisees (configurable via .env : ALLOWED_ADMIN_IPS). C'est une securite en couches : meme si un attaquant devine les identifiants admin, il ne peut pas acceder a l'interface Django admin depuis une IP non autorisee. Ce middleware est active en 3eme position dans la pile de middleware Django.

Securite SSL/HTTPS - Headers supplementaires

En production (DEBUG=False), les headers de securite suivants sont actives :

- SECURE_SSL_REDIRECT = True : Redirige toutes les requetes HTTP vers HTTPS
- SESSION_COOKIE_SECURE = True : Les cookies de session ne sont envoyes qu'en HTTPS
- CSRF_COOKIE_SECURE = True : Les cookies CSRF ne sont envoyes qu'en HTTPS
- SECURE_BROWSER_XSS_FILTER = True : Filtre XSS du navigateur
- SECURE_CONTENT_TYPE_NOSNIFF = True : Empeche le MIME sniffing
- X_FRAME_OPTIONS = 'DENY' : Empeche le clickjacking (iframes)
- SECURE_PROXY_SSL_HEADER : Fait confiance au header X-Forwarded-Proto pour les proxys

Validation des mots de passe

Les 4 validateurs integres de Django sont actives :

- UserAttributeSimilarityValidator : Empeche les mots de passe trop similaires au profil
- MinimumLengthValidator : Longueur minimale requise
- CommonPasswordValidator : Verifie contre un dictionnaire de mots de passe communs
- NumericPasswordValidator : Empeche les mots de passe uniquement numeriques

La validation est aussi appliquee cote API via les serializers DRF (register + change password).

RESUME DES CHOIX TECHNOLOGIQUES

Ce document presente l'ensemble des technologies utilisees pour le developpement de Djezzy Supervision Reseau, un systeme NOC (Network Operations Center) de gestion des reclamations et supervision des sites BTS.

Chaque choix technologique a ete justifie par des criteres concrets : popularite et communaute, qualite technique, securite, productivite, et coherence avec l'ecosysteme du projet. L'architecture 3-tier (React / Django / PostgreSQL) est un standard eprouve pour les applications web professionnelles.

Les technologies open source ont ete privilegiees (Django, PostgreSQL, Leaflet, Firefox, draw.io) pour la transparence, la souverainete des donnees, et l'absence de cout de licence. L'integration de Mistral AI apporte une dimension d'intelligence artificielle unique tout en restant souveraine (entreprise francaise).

5

Langages

12+

Librairies

1

Base de donnees

9

Mesures securite